1. CI/CD 개념 및 활용

1. 백엔드 개발자에게 CI/CD를 요구하는 이유

현재 빠르게 변화하는 디지털 환경에서 고품질 소프트웨어 애플리케이션을 빠르고 신뢰성 있게 제공하는 것은 기업에 필수적입니다. 지속적 통합 및 지속적 배포(CI/CD)는 소프트웨어 개발을 간소화하는 필수적인 관행으로 떠오르며, 효율적인 배달과 최소한의 오류를 보장합니다.

백엔드 개발자에게 CI/CD(지속적 통합/지속적 배포)를 요구하는 이유는, **더 빠르고 안정적인 서비스 개발과 운영**을 위해서입니다. 기업은 단지 코드를 잘 짜는 것을 넘어서, **개발-테스트-배포까지 자동화하고 신속하게 운영할 수 있는 개발자**를 원합니다.

CI/CD는 **단순 자동화 도구가 아니라, 개발자 업무 효율과 서비스 신뢰성을 높이는 필수 역량**이기 때문에, **백엔드 개발자라면 반드시 숙지해야 할 실무 기술**로 요구됩니다.

2. 전통적인 개발 생명 주기

전통적인 개발 생명 주기(Software Development Life Cycle, SDLC)는 소프트웨어 개발을 위한 여러 단계를 포함하고 있습니다. 이 과정은 프로젝트의 성공적인 실행을 보장하고, 품질 높은 소프트웨어를 만들기 위해 체계적인 접근 방식을 제공합니다. 전통적인 SDLC 모델에는 보통 다음과 같은 단계들이 포함됩니다:

1. **다양한 개발 팀들이 서로 다른 코드 리포지토리를 유지**합니다.

   각 개발 팀은 자신이 담당하는 기능이나 모듈에 대해 독립적으로 코드를 작성하고 관리합니다. 이렇게 팀마다 리포지토리를 따로 관리함으로써, 각 팀이 다른 팀의 코드에 영향을 주지 않으면서 개발을 진행할 수 있습니다.

2. **빌드 및 통합 팀이 코드를 통합**합니다.

   여러 개발 팀에서 작성된 코드를 통합하는 역할은 주로 빌드 및 통합 팀이 맡습니다. 이 팀은 각기 다른 리포지토리에서 받은 코드를 하나의 통합된 코드베이스로 합쳐, 전체 시스템이 잘 작동하도록 조정합니다.

3. **통합된 코드를 패키지 형태로 컴파일**합니다.

   통합된 코드는 패키지로 컴파일됩니다. 이 패키지는 애플리케이션을 배포할 준비가 된 형태로 만들어집니다. 즉, 개발과 테스트를 거쳐 실제 운영 환경에 배포될 준비가 된 코드입니다.

4. **패키지와 함께 배포 지침이 운영팀으로 전송됩니다.**

개발팀은 패키지와 함께 배포에 필요한 지침서나 설명서를 운영팀에게 전달합니다. 이 지침서는 패키지가 어떻게 배포될지, 배포 후에 무엇을 해야 하는지 등의 내용을 포함합니다.

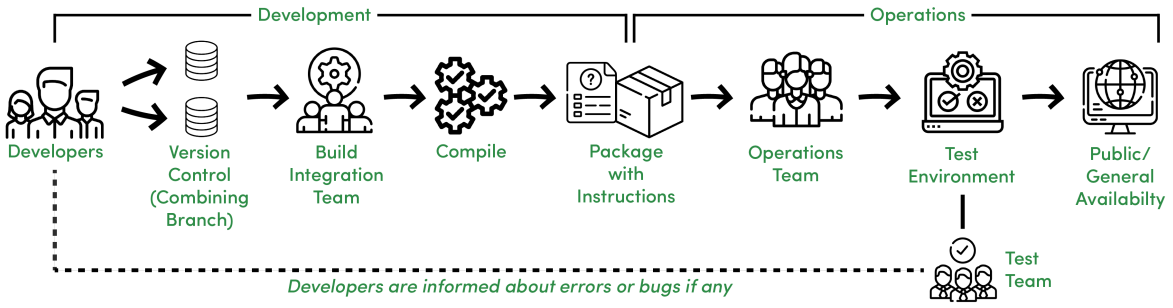
5. **운영팀은 패키지를 테스트 환경에 배포하고, 테스트팀이 패키지를 테스트합니다.**

   운영팀은 받은 패키지를 테스트 환경에 배포하여, 실제 운영에 앞서 여러 가지 환경에서 소프트웨어가 제대로 작동하는지 확인합니다. 그런 다음, **테스트팀**이 해당 패키지를 테스트하여 기능적 요구 사항을 충족하는지, 버그나 오류가 없는지를 검사합니다.

6. **테스트팀은 문제를 발견한 경우 개발팀에 알리고, 문제가 없다면 배포 승인을 합니다.**

   테스트팀은 테스트 결과를 바탕으로 발견된 문제를 개발팀에 전달하여 수정하도록 합니다. 만약 모든 것이 정상적으로 동작하고, 문제를 발견하지 못했다면, 테스트팀은 **생산 환경으로의 배포 승인을** 하게 됩니다.

SDLC의 전통적인 모델은 순차적인 단계들을 따르는 방식이지만, 현대에는 **애자일(Agile)**, **스크럼(Scrum)**, **DevOps** 등 다양한 변형 모델들이 적용되고 있습니다. 이들 방법론은 개발 속도를 빠르게 하고, 반복적인 개발 및 피드백을 강조하여 더 유연한 소프트웨어 개발을 가능하게 합니다.



3. CI/CD란 무엇인가?

전통적인 개발 방식 단점

전통적인 소프트웨어 개발 모델, 특히 **워터폴(Waterfall) 모델**처럼 각 개발 단계가 순차적으로 진행되는 방식은, 빠르게 변화하는 시장과 기술 환경에서는 **효율적이지 않을 수** 있습니다. 워터폴 모델에서는 전체 프로젝트가 끝날 때까지 실제로 기능하는 소프트웨어를 볼 수 없기 때문에, 개발 주기가 길고, 중간 피드백을 받을 수 없다는 단점이 있습니다. 이로 인해 다음과 같은 문제가 발생할 수 있습니다:

- **오랜 개발 주기**: 제품을 개발하고 완성하기까지 긴 시간이 소요됩니다. 이 기간 동안 변경 사항이나 새로운 요구 사항에 대응하기 어려울 수 있습니다.

- **낮은 유연성**: 제품이 완성될 때까지 실제로 어떤 부분이 잘못되었는지 알 수 없습니다. 테스트와 피드백이 마지막 단계에서만 가능하여, 문제를 나중에 발견하는 경우 수정하기가 매우 어렵고 비효율적입니다.

CI/CD가 등장하는 이유

이런 한계를 극복하기 위해 **CI/CD**(지속적인 통합 및 지속적인 배포/배달) 방식이 점점 더 중요해지고 있습니다. **CI/CD**는 단순히 **도구**의 문제가 아니라, **개발 문화와 실천**을 바꾸는 개념입니다.

CI/CD 정의:

- **CI/CD**는 **지속적인 통합(Continuous Integration)**과 **지속적인 배포 (Continuous Delivery)** 또는 **지속적인 배포(Continuous Deployment)**의 약어입니다.
- 이는 **공유 리포지토리에서 통합된 코드를 사용하여, 소프트웨어를 하루에도 여러 번 자동으로 프로덕션에 배포하는** 과정입니다.

지속적인 통합(Continuous Integration)

- **지속적인 통합**(CI)은 개발자들이 하루에도 여러 번 **공유된 리포지토리**에서 **자동화된 프로세스**를 통해 코드를 통합하는 방식입니다.
- 이를 통해 모든 개발자들은 각자의 코드 변경을 **자주** 그리고 **작게** 통합하여, 충돌을 미리 해결하고, 코드의 품질을 **지속적으로 점검**할 수 있습니다.
- 예를 들어, 개발자가 코드를 커밋할 때마다 자동으로 빌드하고 테스트를 진행하여, 통합된 코드가 제대로 작동하는지 빠르게 확인할 수 있습니다.

지속적인 배포(Continuous Delivery) 또는 지속적인 배포(Continuous Deployment)

- *지속적인 배포(Continuous Delivery)**는 소프트웨어가 **자동으로 테스트 환경이나 프로덕션 환경에 배포되는** 과정을 의미합니다.
- *지속적인 배포(Continuous Deployment)**는 지속적인 배포의 한 단계 더 발전한 형태로, 자동화된 테스트를 통과한 소프트웨어가 **수동 개입 없이 자동으로 프로덕션 환경에 배포**되는 방식입니다.

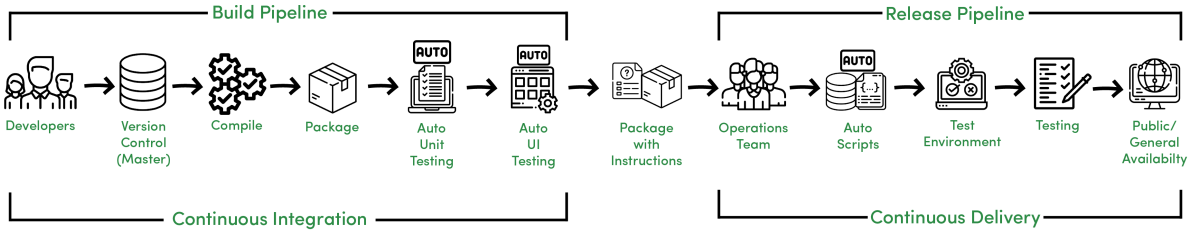
CI/CD가 제공하는 주요 이점

1. **빠른 피드백**: 코드를 지속적으로 통합하고 배포하기 때문에, 기능이나 버그가 발견되면 빠르게 대응할 수 있습니다. 이로 인해 개발자들은 실시간으로 문제를 해결할 수 있습니다.

2. **지속적인 배포와 빠른 출시**: 소프트웨어가 자동으로 배포되므로, 작은 기능 변경이나 버그 수정을 신속하게 사용자에게 제공할 수 있습니다. 제품이 **빠르게 시장에 출시**되고, **사용자 피드백**을 반영하기 쉽습니다.
3. **품질 향상**: 자동화된 테스트와 통합이 이루어지므로, 코드의 품질을 지속적으로 점검하고 향상시킬 수 있습니다. 이는 버그를 사전에 예방하고, 안정적인 소프트웨어를 제공합니다.
4. **효율적인 협업**: 개발자들이 같은 코드베이스에서 작업하므로, **협업이 효율적으로 이루어지며**, 통합 및 배포가 자동화되어 팀 간의 커뮤니케이션이 원활해집니다.

CI/CD의 핵심 원칙

1. **자동화**: 빌드, 테스트, 배포 등의 모든 프로세스를 자동화하여 반복적인 작업을 줄이고, 인적 오류를 최소화합니다.
2. **지속적인 통합**: 코드 변경 사항을 자주 통합하여 코드베이스를 항상 최신 상태로 유지합니다.
3. **빠른 배포**: 코드를 빠르게 배포하여, 기능 추가 및 수정이 빠르게 이루어질 수 있도록 합니다.
4. **빠른 피드백**: 자동화된 테스트와 통합을 통해 실시간으로 피드백을 제공하여, 문제를 초기에 발견하고 수정합니다.



** 이점 요약

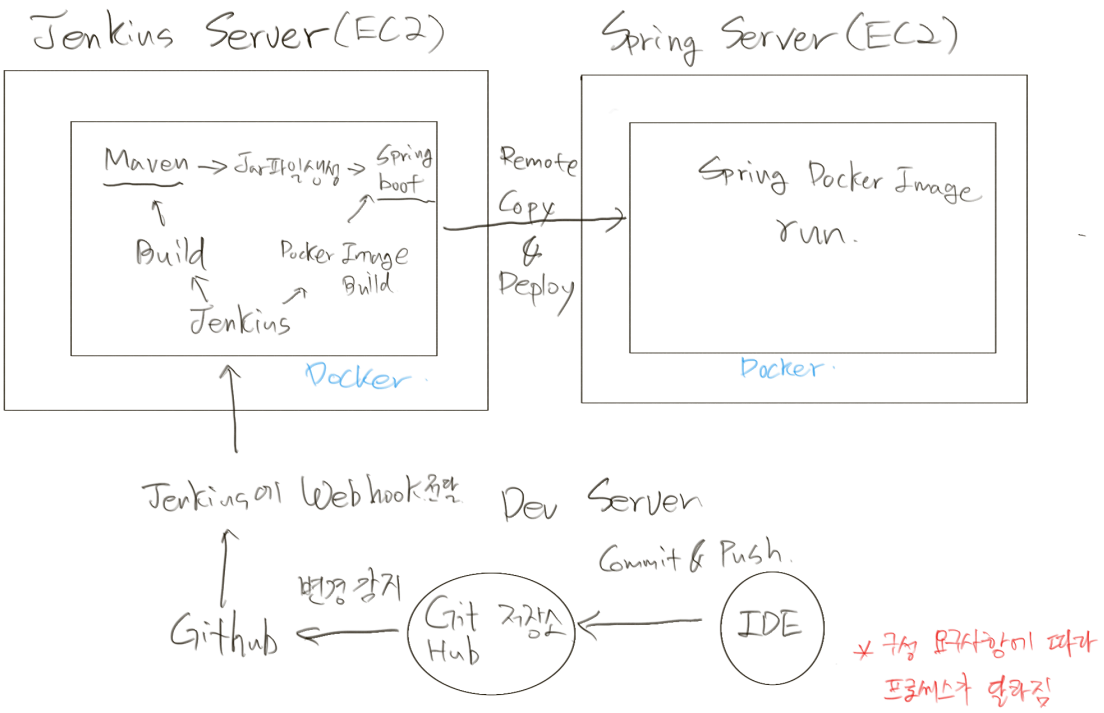
이점	설명
속도	코드 수정 → 배포까지 짧은 주기
안정성	자동 테스트 + 배포로 실수 방지
품질	코드 검증 자동화
협업	여러 명의 코드 쉽게 통합
피드백	문제 발견 → 수정 → 반영까지 빠름

5. CI/CD 환경 소개

1. 버전 관리: Git, GitHub
2. 개발 환경(온프레미스):
    - 백엔드 서버(Windows OS): Java, Spring Boot, WAS(Web Application Server)
    - DB 서버(Windows OS): MySQL
    - 프론트엔드(Windows OS): React, Nginx (본 수업에선 생략)
3. 운영 환경(클라우드):
    - 백엔드(Amazon Linux): Docker, Java, Spring Boot, WAS(Web Application Server)
    - DB 서버(Amazon RDS): MySQL
    - 프론트엔드(Amazon Linux): React, Nginx (본 수업에선 생략)
    - Jenkins 서버(Amazon Linux) : Docker, Jenkins, Maven
4. 배포 환경 일관성 가상화 플랫폼 : Docker
5. 빌드 자동화 도구 : Maven
6. CI/CD 파이프라인 구축 : Jenkins

5. CI/CD를 활용한 배포 자동화 프로세스

- 개발 서버, 젠킨스 서버, Spring 서버(운영 서버) 구성 흐름도



- 개발 및 배포 프로세스
  1. **개발자가 기능을 개발하고 Git Push 수행**
     - GitHub에서 Pull Request를 만들고, 코드 리뷰 후 `main` 혹은 `develop` 브랜치에 머지.
  2. **GitHub Webhook을 통해 Jenkins에서 빌드 트리거**
     - Jenkins가 **Webhook**을 감지하여 파이프라인을 시작하고, 빌드를 실행.
  3. **Jenkins에서 스크립트를 이용해 JAR 파일 빌드 및 자동화된 테스트**
     - 단위 테스트, 통합 테스트, 코드 품질 분석 등을 실행. (본 수업에선 생략)
     - 테스트가 성공하면 JAR 파일을 빌드하여 Docker 이미지로 패키징.
  4. **Jenkins에서 Docker 이미지 빌드 및 배포**
     - 빌드된 Docker 이미지를 Docker Hub 혹은 자체 Registry로 푸시.
     - 운영 서버에서 Docker 이미지를 Pull하여 배포.
  5. **운영 서버에서 Docker 컨테이너 실행 및 Spring 앱 운영**
     - Docker Compose나 Kubernetes로 여러 서버 및 서비스를 관리.(본 수업에선 생략)
  6. **자동 배포 및 기능 활성화**

- 새 기능이 개발 서버에서 Git Push 후 자동으로 운영 서버에 배포.
- 배포 후 **모니터링**과 **로깅**을 통해 애플리케이션의 상태를 추적. (본 수업에선 생략)

7. **배포 실패 시 자동 롤백**
    - 배포 실패 시 **자동 롤백** 기능을 통해 이전 안정 버전으로 돌아갈 수 있도록 설정. (본 수업에선 생략)